Available online at www.sciencedirect.com

Manufacturing Letters

Manufacturing Letters 41 (2024) 1728–1733

52nd SME North American Manufacturing Research Conference (NAMRC 52, 2024)

Large Language Models Empower the Reliability of Disassembly in Remanufacturing

Liqiao Xia[a], Jiazhen Pang[a], Chengxi Li[a], Ruoxin Wang[a], Pai Zheng[a,*]

[a]*Hong Kong Polytechnic University, HKSAR, China*

Abstract

Disassembly is a crucial stage in the product lifecycle, significantly contributing to remanufacturing, waste minimization, and resource recovery by facilitating the reuse and recycling of components and materials. However, engineers often face a plethora of choices during the disassembly process, each linked to uncertain outcomes, leading to substantial challenges. To address this, we introduce an assembly sequence evaluation model offering a quantitative analysis for diverse conditional decisions. Initially, an event graph is devised to outline the hierarchical structure, effectively shrinking the sequence space through the utilization of engineering semantics. Subsequently, a reinforcement learning model is established, with the reward function defined by large language models that use tailored prompts for their sequences. Deep Q-learning is then applied to train the reinforcement learning model, incorporating a selected set of ground truth sequences to highlight the correct one. Finally, a case study on a decelerator disassembly is presented to illustrate the effectiveness of the proposed method.

Keywords: Disassembly; Reinforcement learning; Large language models, Remanufacturing, Graph neural network.

1. Introduction

Remanufacturing has emerged as an indispensable strategy in contemporary manufacturing paradigms, signaling a pivot towards sustainable and circular economic modalities [1]. This transformative process rejuvenates used products into functional entities, thereby conserving resources and attenuating environmental impact. It imparts a second life to discarded or obsolete products, extending their utility and constricting the demand for virgin raw materials [2].

The critical inaugural step in remanufacturing is disassembly, the process that disentangles a product into its constituent components [3]. This facilitates the recovery and reutilization of valuable components, hence curtailing raw material demand, diminishing manufacturing costs, and mitigating environmental repercussions. In essence, disassembly serves as a conduit to efficacious remanufacturing, metamorphosing waste into potential wealth. Furthermore, it provides a platform to inspect and evaluate the condition of used products, a vital aspect for quality control in the remanufacturing process [4].

Historically, the orchestration and sequence of disassembly tasks relied heavily on the expertise and intuition of engineers [5]. However, this method possesses inherent limitations, as it may not invariably yield the most competent or efficacious outcomes. To circumvent these constraints, knowledge-based models have been conceived, employing explicit rules and heuristics extracted from expert knowledge to navigate the disassembly process [6]. Although these models can deliver more systematic and consistent results compared to experience-based approaches, they may lack the exibility or adaptability to manage intricate or unforeseen situations.

Inherently, disassembly frequently encompasses unpredictability and uncertainty [7]. Engineers may confront unfamiliar equipment, necessitating an understanding of its structure and the optimal disassembly methods. Their experience, while beneficial, may not always offer reliable guidance due to the

* Corresponding author. Tel.: +852 2766 5633
E-mail address: pai.zheng@polyu.edu.hk (Pai Zheng).

novelty and complexity of the task. In these instances, engineers often seek advice or references to inform their decisions, but finding trustworthy sources of information can be challenging. Traditional knowledge databases or manuals might not encompass specific or atypical situations, and peers or experts might not always be accessible for consultation [8]. Furthermore, the applicability of available advice may uctuate depending on the specific context and conditions. This uncertainty and dearth of reliable information sources highlight the demand for more robust and adaptable tools in disassembly planning.

The advent of Large Language Models (LLM) has garnered global interest [9]. LLM can also contribute to disassembly planning, as the disassembly sequence is replete with semantic information. Queries to LLM can provide alternative perspectives on this sequence, although the responses may sometimes be uncertain [10]. The challenge lies in harnessing the knowledge embedded in LLM to aid disassembly sequence planning, which remains an unresolved task.

To address these challenges, we propose a novel LLM-based Reinforcement Learning method for evaluating disassembly sequences, offering quantitative analyses for a variety of potential sequences. Specifically, the primary contributions of this work can be encapsulated as follows:

1) 'A disassembly graph is proposed to structure various disassembly events in a hierarchical format, thereby providing the extra calculation space for the downstream task.

2) A compressed Relational Graph Convolutional Network (RGCN) is presented to vectorize the nodes in the disassembly graph.

3) A Reinforcement Learning (RL) model is introduced to derive quantitative analysis for the disassembly sequence, which works in tandem with LLM to serve as a reward function, and Recurrent Neural Network (RNN) as prediction model.

2. Methodology

2.1. Problem Statement

This work aims to generate a robust and effective disassembly sequence facing unfamiliar task, as shown in Fig. 1. While a disassembly graph delineates dependent disassembly relationships, numerous potential sequences remain, with the optimal one remaining elusive. To find the optimal disassembly sequence, first, the disassembly graph is constructed to provide basic instructions to the disassembly task. However, this disassembly graph may indicate many possible sequences. How to select the optimal one is the concern of the following tasks. Here, a reinforcement learning (RL) is proposed to generate the feasible sequence with the evaluation of LLM. The disassembly sequence will be injected into RNN to predict the action as well as the whole sequence. Then this whole sequence will be evaluated by LLM accompanying with reward to train the RL model. Under this circumstance, the well-train RNN can generate the optimal disassembly sequence.

2.2. Disassembly Graph Establishment

Our methodology involves constructing a disassembly graph to visualize the disassembly process, as shown in Fig. 2. This graph considers the hierarchical structure of the components, their relationships, and the sequence of operations [11]. The notations used are as follows:

Nodes N: Represent the components in the assembly. Each node is assigned a level (L) based on its position in the hierarchy. Edges E: Represent the relationships between components. There are two types of edges: Solid edges E_s: Represent contact relationships. Dashed edges E_d: Represent non-contact relationships.

2.2.1. Hierarchical Structure of Components

In our graph model $G(N, E)$, components N are organized hierarchically. Each component is assigned a level L based on its dependencies. Higher-level components, which depend on the disassembly of lower-level ones, are placed higher in the hierarchy. This tiered structure dictates the sequence of the disassembly process.

2.2.2. Contact and Non-Contact Relationships

The graph also portrays the contact and non-contact relationships between components through the use of edges E. Contact relationships, represented by solid edges E_s, imply a physical connection between components. A component linked by a solid edge must be removed before or simultaneously with its counterpart.

Non-contact relationships, represented by dashed edges E_d, signify logical connections between components that do not physically interact but are indirectly related through a sequence of disassembly operations. Understanding non-contact relationships can reveal hidden dependencies and potential bottlenecks in the disassembly process.

2.2.3. AND and OR Precedence

The graph model incorporates AND and OR precedence relationships. AND precedence exists when two or more components can be disassembled concurrently. These are represented by parallel edges originating from the same node in the graph. Meanwhile, OR precedence is represented when there are alternative paths from one node to others, denoting multiple feasible sequences for disassembling components. Fig. 2 Provided an example of disassembly graph. This disassembly graph can provide fundamentals for the following graph embedding training.

2.3. Relational Graph Convolution Network-Based Embedding

The essence of GCN is that: the association of each node is regarded as prior knowledge, with the goal of improving the model's performance and reducing the training time.

For undirected graphs G=(V, ξ), where V represents finite nodes. The ξ is a set of edges. For two nodes v_i,v_j in a graph, the adjacent matrix is A_{ij}.

The Graph Convolutional Network (GCN) represents a specific category of Graph Neural Networks (GNNs). It strives to

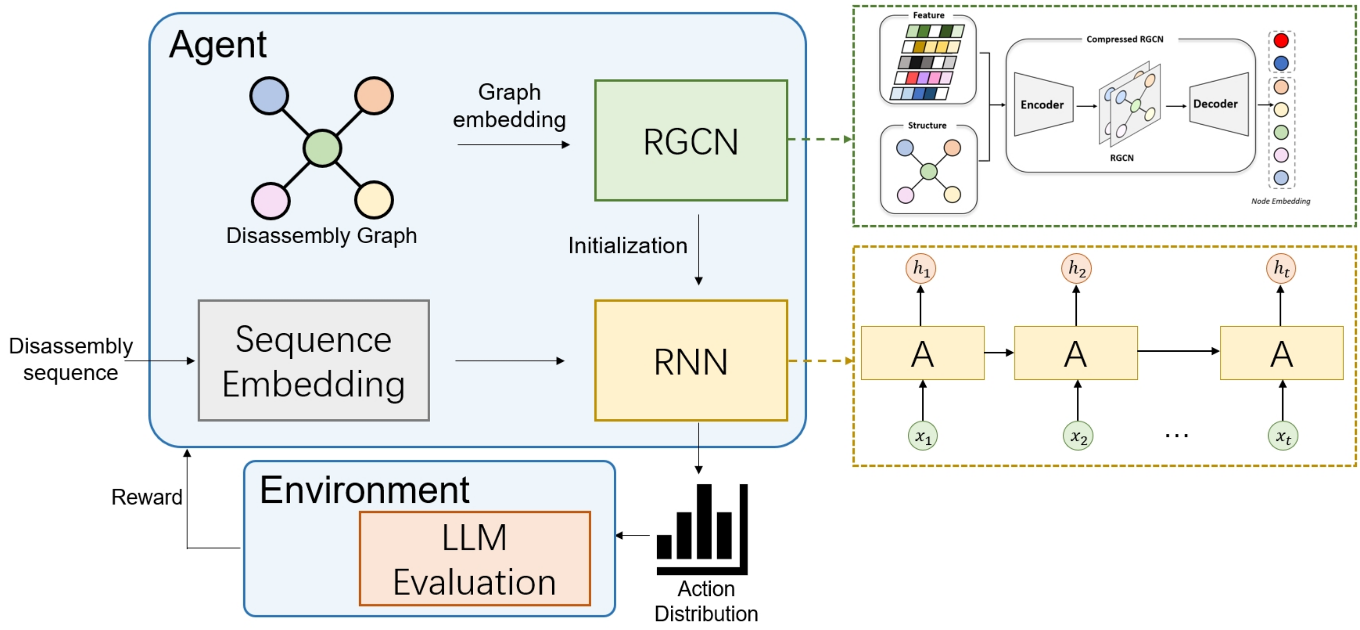

Figure 1. Arichitecture of Proposed Model

learn more dependable representations through the utilization of a convolutional kernel, a process which is facilitated by the underlying graph structure:

$$H^{(k+1)} = f(H^k, A), \tag{1}$$

herein, k denotes the layer number, while H^k signifies the representation within the k^{th} layer. It is important to note that $H^{(0)}$ is equivalent to x. The function f symbolizes the generation of a new node representation, derived from the aggregation of its neighboring nodes.

$$H^{(k+1)} = f(H^k, A) = \sigma(AH^kW^k), \tag{2}$$

where W^k $\mathbb{R}^{F^{k-1}*F^k}$ represents the weight matrix, with F^k denoting the dimensionality of the node embedding in the k^{th} layer. The activation function is represented by σ. Equation 2 facilitates the aggregation of representations from neighboring nodes.

$$(AH)_i = A_iH = \sum_j A_{ij}H^j, \tag{3}$$

where nodes v_i and v_j are connected, the adjacency matrix element A_{ij} equals 1. Consequently, the new representation of node v_i is derived from the summation of its neighboring nodes. Equation 3 presents a basic, yet direct approach to execute graph-level message passing. Nonetheless, this method is not without its shortcomings. Firstly, it consolidates representations from neighboring nodes while neglecting the node itself. Moreover, the features of nodes with high degrees proliferate during the summarization procedure, while those of nodes with low degrees diminish. This scenario can lead to gradient explosion or vanishing during network training. Aiming at the first limitation, each node adds a self-loop mechanism, changing the adjacent matrix:

$$\widetilde{A} = A + I_N, \tag{4}$$

where I_N represents the identity matrix for N*N dimension.

Regarding the second limitation, normalization of the adjacency matrix can be implemented such that the summation of each row equals 1. Specifically, the adjacency matrix can be multiplied by an inverse degree matrix as follows:

$$H^{(k+1)} = f(H^k, A) = \sigma(\widetilde{D}^{-1}\widetilde{A}H^kW^k), \tag{5}$$

where D indicates degree matrix, and $\widetilde{D_{ii}} = \sum_{j=1} \widetilde{A_{ij}}$. Furthermore, Eq. 5 can be improved by a symmetric normalization to considerate the degree of neighbour nodes:

$$H^{(k+1)} = f(H^k, A) = \sigma(\widetilde{D}^{-\frac{1}{2}}\widetilde{A}\widetilde{D}^{-\frac{1}{2}}H^kW^k). \tag{6}$$

The disassembly graph, being a heterogeneous graph with multiple edge types, surpasses the capabilities of a conventional GCN. Accordingly, GCN must be expanded into a Relational Graph Convolutional Network (RGCN) by distinctly accounting for message passing across various relations, as suggested by Schlichtkrull et al. (2018) [12]. This can be computed as follows:

$$H^{(l+1)} = \sigma\left(\sum_{r=1}^{R} \widetilde{D_r}^{-\frac{1}{2}} \widetilde{A}_r \widetilde{D_r}^{-\frac{1}{2}} H^{(l)} W_r^{(l)}\right), \tag{7}$$

where R represents the number of edge types. For a particular type r, the corresponding adjacency matrix, degree matrix, and weight matrix are represented by $\widetilde{A}_r$, $\widetilde{D_r}$, and $W_r^{(l)}$, respectively. The extended message passing rule delineates how data within a relational network should interact with adjacent nodes. During the message-passing stage, the embedding is accumulated across numerous relations. Unlike the GCN, these relation-specific transformations furnish a unique representation space for each type of node embedding. Additionally, the summarization process aligns multiple types of node spaces. Each type of edge connected to a specific node will independently aggregate their

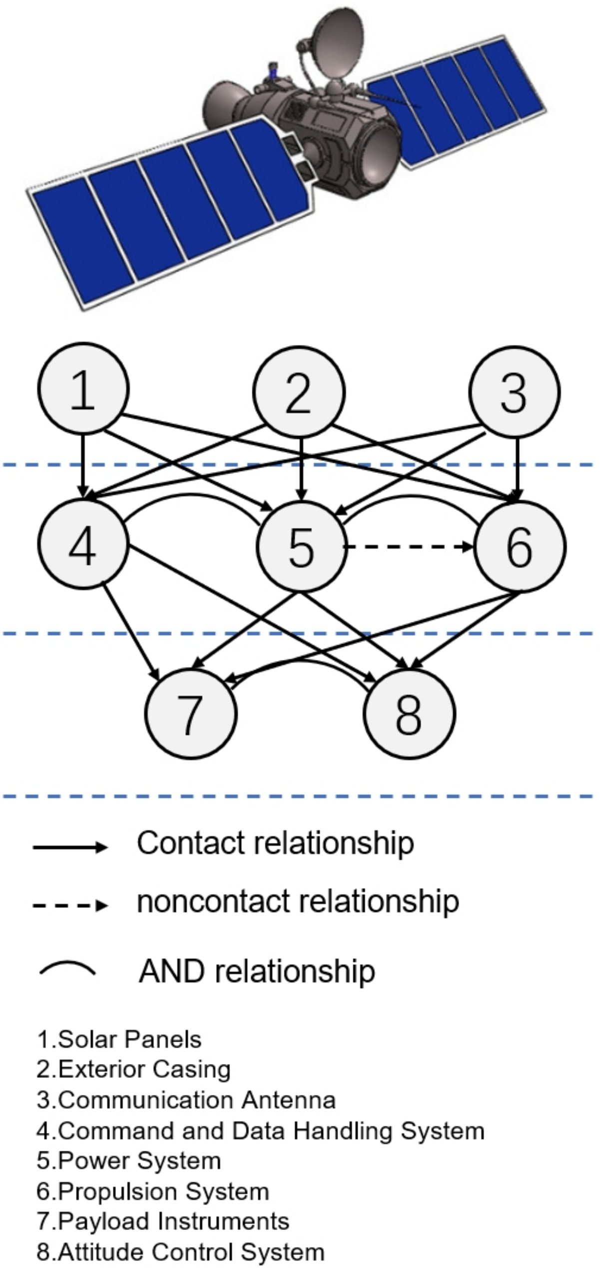

Figure 2. Disassembly graph of satellite

associated connected nodes. These disparate types of aggregations will then be combined with the node itself (for instance, through a fully connected layer) to yield an optimized node embedding.

Taking inspiration from the AutoEncoder, this paper utilizes RGCN modules to increase the density of node features. The Encoder portion of the RGCN modules reduces the dimensionality of the node feature. Each Encoder module sequentially incorporates an RGCN, batch normalization, a ReLu activation function, and a dropout function (excluding the dropout in the final module). The RGCN module corresponds to Equation 7, following a dimension reduction process. For example, if the dimension of $H^{(l)}$ is 256, the dimension of $H^{(l+1)}$ would be 128.

Therefore, the Encoder process can be restructured based on Equation 7 as follows:

$$H^{(l+1)} = f\left(\sigma\left(\sum_{r=1}^{R} \widetilde{D_r}^{-\frac{1}{2}} \widetilde{A_r} \widetilde{D_r}^{-\frac{1}{2}} H^{(l)} W_e^{(l)}\right)\right), \tag{8}$$

where f indicates the Encoder module. On the contrary, the Decoder process has opposite logic, where the dimension will be increased:

$$\hat{H}^{(l-1)} = f\left(\sigma\left(\sum_{r=1}^{R} \widetilde{D_r}^{-\frac{1}{2}} \widetilde{A_r} \widetilde{D_r}^{-\frac{1}{2}} \hat{H}^{(l)} W_d^{(l)}\right)\right), \tag{9}$$

where $\hat{H}^{(l)}$ signifies the l^{th} layer in the Decoder process, while f denotes the Decoder module. It's noteworthy that the Encoder and Decoder share an equal number of modules. As a result, the subsequent module in the Decoder is labeled as $l-1$ rather than $l+1$. Ultimately, $\hat{H}^{(1)}$ represents the final output of the Decoder module.

Once the node embedding is well trained, it will serve as the initial embedding in the sequence prediction model in RNN.

2.4. Multiobject Deep Reinforcement Learning-Based Disassembly Sequence Planning

This section introduces the Deep RL (DRL) architecture of sequence planning based on the Proximal Policy Optimization (PPO) algorithm [13].

DRL aims to train an agent to take actions that maximize the cumulative reward, gathered through interaction with the environment. For the sequence planning problem, DRL can facilitate real-time scheduling and strategy alteration based on the current environmental state, thus effectively handling emergencies. In comparison to value-based methods, policy-based procedures are more apt for problems involving continuous state and action spaces. Given the high-dimensional complexities of state and action spaces resulting from an increased scale of the sequence planning problem, policy-based methods become necessary. Among these, the PPO algorithm stands out. By limiting policy updates, it reduces sensitivity to parameter settings, thereby delivering superior performance. Hence, this paper employs the PPO algorithm to address the job-shop scheduling problem.

In RL, a learning agent learns to map situations to actions by interacting with its environment over time, thereby maximizing the rewards accumulated. Specifically, at each time step t, the agent observes a state s_t from a discrete state space S. An action $a_t \quad AC$ is then selected based on the agent's policy π, leading to a transition in the environment to a new state s_{t+1} and generating an immediate reward r_t. In addition, it also yields a numerical reward r_{t+1}, which is used to update the policy π.

In this study, PPO is utilized to train the RL model. Policy gradient methods optimize the policy by updating parameters in the direction that maximizes the objective function:

$$\Theta =: \Theta + n * \quad {}_{\Theta}L(\Theta) \tag{10}$$

where Θ is the model parameters, n is the learning rate, and $L(\Theta)$ is the loss function of expectation of reward, which can be defined as:

$$L(\Theta) = \widehat{E}_{s\sim d^{\pi}}[\sum_{a\ A} \pi_{\Theta} A^{\pi}(s, a)] \tag{11}$$

When utilizing the original policy gradient method for parameter updates, minor alterations in the parameter space often induce significant shifts in the policy space, resulting in unstable training. As a countermeasure, the PPO algorithm simplifies this process by employing a clipped surrogate objective. PPO updates the parameters of the RNN [14] and the policy only when the current representation and the policy module result in a local improvement of scheduling performance. More specifically, the clipped surrogate objective is defined as follows:

$$L^{CLIP}(\Theta) = \widehat{E}_r[min(r(\Theta)\hat{A}_t, clip(r(\Theta), 1-\epsilon, 1+\epsilon)\hat{A}_t)], \tag{12}$$

where $r_{\Theta} = \frac{\pi(a\ g;\Theta_{new})}{a\ g;\Theta_{old}}$, and the function *clip* limits $r(\Theta)$ within [1-ϵ,1+ϵ], and $\hat{A}_t$ refers to the advantage estimate at time t.

2.5. Disassembly Sequence Evaluation Based on Large Language Models

Disassembly sequences carry a wealth of semantic information that can be integrated with Large Language Models (LLM). LLMs can supply additional practical knowledge based on customized prompts. For instance, a prompt might inquire, ”Assume you are an expert in disassembly. Currently, I’m disassembling a turbofin and my disassembly sequence is XXX. If you could rate the reasonability of this disassembly sequence on a scale from 0 to 100, what score would you assign?” It is noted that ’reasonability’ may be substituted with other specific terms depending on various factors, including time consumption, human economic costs. Subsequently, the LLM provides a score based on its internal mechanism. Generally, an LLM operates like a ”black box” with no explicit explanation. However, it is trained on a broad corpus encompassing extensive engineering knowledge, much of which may be unknown to many engineers.

In this study, the score derived from the LLM will function as the environmental reward within the RL model, thereby acting as one of the objectives.

3. Case Study

In this study, a disassembly of the gearbox will be used to study, as shown in Fig. 3. There are six components need to be disassembled: ’Front bearing washer’, ’Front bearing’, ’Back bearing washer’, ’Back bearing’, ’Axis’, and ’Gear’.

3.1. Reinforcement Learning Setting

Reward: The reward function is composed of two parts: the ground truth reward and the Large Language Model (LLM) reward.

Observation: This refers to the state of the gearbox during disassembly.

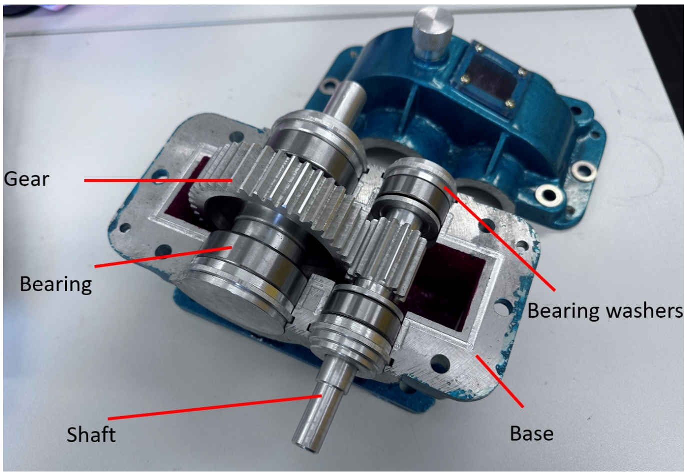

Figure 3. Gearbox structure

ActionS pace: This encompasses the connected nodes in the disassembly graph.

With these settings as input, the proposed method can be utilized to find the optimal solution for the task at hand. In this scenario, the agent environment is handled by the gym package, which is a python package serving for the simulation environment [15].

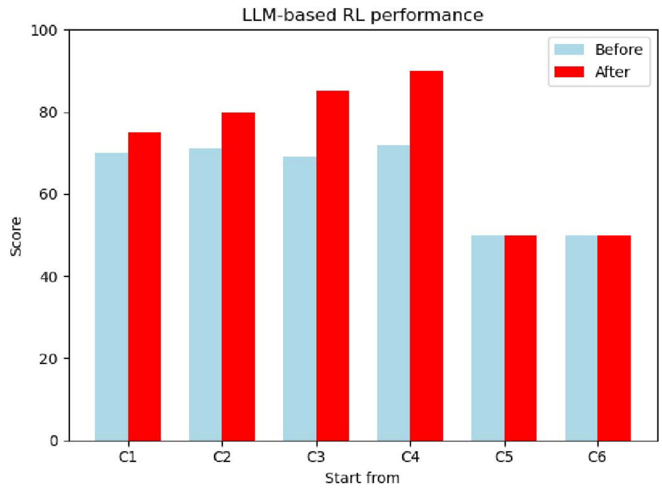

Figure 4. Sequence Evaluation with RL

3.2. LLM Data preprocessing

As described in Section 2.5, the disassembly sequence will be assessed by a Large Language Model (LLM). In this experiment, we utilize ’gpt-3.5-turbo’ as the language model, and an OpenAI API is used to invoke this function to generate valid evaluation content for the potential disassembly sequence.

The semantic response content may vary in style, necessitating score extraction based on semantic logic and keywords. For example, phrases such as ’90 out of 100’ or ’scores 90’ indicate the score. Therefore, a set of rules is implemented to extract the score from the response content due to the various formats of estimated value in the answers.

3.3. Sequence Evaluation

In this scenario, there are six candidate components, leading to six different situations each starting from a distinct component. The results are depicted in Fig. 4, where $C1$ to $C6$ represent the six different components. The motivation behind this arrangement stems from the vast number of possible sequences. Presenting the scores of all these sequences would be impractical. Therefore, categorizing these sequences based on their first component allows for a simpler interpretation of the proposed method's contribution. Specifically, $C1$ is the 'Front bearing washer', $C2$ is the 'Front bearing', $C3$ is the 'Back bearing washer', $C4$ is the 'Back bearing', $C5$ is the 'Axis', and $C6$ is the 'Gear'. 'Before' denotes the average score of the sequence starting from this value, while 'After' signifies the score of the sequence generated by the proposed model. It is noted that the last two bars show a score of 50. This is because the lowest evaluation score from GPT is 50, regardless of how unexpected the sequence may be.

As illustrated in Fig. 4, the 'After' scores are higher than the 'Before' scores for the first four components. The last two, however, remain constant. This is due to the fact that it is not feasible to disassemble the final two components without disassembling the first four. Overall, the proposed model can generate a superior disassembly sequence compared to random selection. This experimental result demonstrates that reinforcement learning can capture the hidden relationships within the disassembly graph to achieve predefined objectives.

4. Conclusion

Disassembly plays a critical role in remanufacturing, yet engineers often encounter unforeseen scenarios and a multitude of uncertainties, even when they can identify all relevant components. Hence, there's a pressing need for explainable and reliable disassembly sequences. To tackle this challenge, this study utilized a LLM as a reward function in RL to generate feasible sequences based on observed situations. Here, the LLM provides a reliable evaluation to refine the RL model.

Looking forward, several avenues for development exist. Firstly, the backbone RNN model could be substituted with other time series models to better capture sequence information. Secondly, other aspects such as cost and environmental friendliness could be considered when determining the disassembly sequence. Third, a larger disassembly graph is anticipated to evaluate the robustness of proposed method.

Acknowledgements

This research is partially funded by the Department of Science and Technology of Guangdong Province (GDSTC) (Project no.: 2023A1515011557), the Mainland-Hong Kong Joint Funding Scheme (MHX/001/20), Innovation and Technology Commission (ITC), Hong Kong Special Administration Region.

References

[1] M. Kerin and D. T. Pham, "Smart remanufacturing: a review and research framework," *Journal of Manufacturing Technology Management*, vol. 31, no. 6, pp. 1205–1235, 2020.

[2] S. Yang, N. Nasr, S. Ong, and A. Y. Nee, "Designing automotive products for remanufacturing from material selection perspective," *Journal of Cleaner Production*, vol. 153, pp. 570–579, 2017.

[3] F. J. Ram´rez, J. A. Aledo, J. A. Gamez, and D. T. Pham, "Economic modelling of robotic disassembly in end-of-life product recovery for remanufacturing," *Computers & Industrial Engineering*, vol. 142, p. 106339, 2020.

[4] G. Tian, M. Zhou, and P. Li, "Disassembly sequence planning considering fuzzy component quality and varying operational cost," *IEEE Transactions on Automation Science and Engineering*, vol. 15, no. 2, pp. 748–760, 2017.

[5] K. Laitala, I. G. Klepp, V. Haugrønning, H. Throne-Holst, and P. Strandbakken, "Increasing repair of household appliances, mobile phones and clothing: Experiences from consumers and the repair industry," *Journal of Cleaner Production*, vol. 282, p. 125349, 2021.

[6] S. Smith, L.-Y. Hsu, and G. C. Smith, "Partial disassembly sequence planning based on cost-benefit analysis," *Journal of Cleaner Production*, vol. 139, pp. 729–739, 2016.

[7] Y. Gao, S. Lou, H. Zheng, and J. Tan, "A data-driven method of selective disassembly planning at end-of-life under uncertainty," *Journal of Intelligent Manufacturing*, pp. 1–21, 2021.

[8] X. Guo, M. Zhou, A. Abusorrah, F. Alsokhiry, and K. Sedraoui, "Disassembly sequence planning: a survey," *IEEE/CAA Journal of Automatica Sinica*, vol. 8, no. 7, pp. 1308–1324, 2020.

[9] E. Kasneci, K. Seßler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer, U. Gasser, G. Groh, S. Günnemann, E. Hüllermeier, *et al.*, "Chatgpt for good? on opportunities and challenges of large language models for education," *Learning and individual differences*, vol. 103, p. 102274, 2023.

[10] M. Yasunaga, J. Leskovec, and P. Liang, "Lm-critic: Language models for unsupervised grammatical error correction," *arXiv preprint arXiv:2109.06822*, 2021.

[11] X. Zhao, C. Li, Y. Tang, and J. Cui, "Reinforcement learning-based selective disassembly sequence planning for the end-of-life products with structure uncertainty," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 7807–7814, 2021.

[12] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. v. d. Berg, I. Titov, and M. Welling, "Modeling relational data with graph convolutional networks," in *European semantic web conference*, pp. 593–607, Springer, 2018.

[13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[14] H. Salehinejad, S. Sankar, J. Barfett, E. Colak, and S. Valaee, "Recent advances in recurrent neural networks," *arXiv preprint arXiv:1801.01078*, 2017.

[15] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," 2016.